

Giro3D Viewer Integration & Management Guide

Introduction

Welcome to the Giro3D Viewer documentation!

To keep the architecture as simple, scalable, and independent as possible, the Polygon platform integrates this 3D viewer using an **Iframe approach**.

Why an Iframe?

By using an iframe, the 3D Viewer operates entirely independently from the main Angular codebase.

1. **Zero Database Cost:** The viewer doesn't need its own server or database. It runs purely on AWS S3.
2. **Decoupled Updates:** You can update the Angular website without worrying about breaking the 3D viewer, and you can update the 3D viewer without touching the Angular code.
3. **Dynamic Loading:** The Angular app simply fetches a URL from its database and injects it into the iframe, automatically rendering the correct 3D terrain for each client!

The Folder Structure (AWS S3)

To keep massive datasets separate from the lightweight web code, your S3 bucket (`polygon-giro3d`) is organized into two distinct folders:

- **/projects/ (The Data):** This contains the giant gigabyte `.tif` files for each client (e.g., `/projects/Chattibhariatu/DTM.tif`). You drag and drop these manually.
- **/assets/ (The Code):** This is generated automatically by Vite when you build the project. It contains all the Javascript and CSS. You **never** touch this folder manually.

How to Add a New 3D Project

When you receive new Cloud-Optimized GeoTIFFs (a DEM/DSM/DTM and an Orthophoto) for a new client, follow these exact steps to add them to the website:

Step 1: Upload the Data to S3

1. Open your AWS S3 Console and navigate to the `polygon-giro3d` bucket.
2. Open the `projects` folder.
3. Create a new folder for your client (e.g., `Ambuja`).
4. Upload your two `.tif` files into this folder. *(Do not rename them after uploading!)*

Step 2: Get the Mathematical Coordinates

Giro3D needs to know the exact mathematical bounding box (`extent`) and the minimum elevation (`zOffset`) of the TIFF files to render them in 3D space.

1. Place a copy of the new `.tif` files in your local `temp/viewer/projects/Ambuja` folder.
2. Open the `check_rasters.py` file and update the `files` array at the top to point to your new TIFF files.
3. Open your terminal and run the script:

```
.\venv\Scripts\python.exe check_rasters.py
```

4. The script will print out the exact `extent` array and `zOffset` number you need. Leave the terminal open so you can copy these!

Step 3: Add the Project to the Code

1. Open `ser.js` in your code editor.
2. Scroll to the `projectsConfig` dictionary (around line 166).
3. Copy and paste an existing project block, rename the key (e.g., `"ambuja_project"`), and update the URLs, `extent`, and `zOffset` using the numbers the Python script just gave you.

```
"ambuja_project": {
  elevationUrl: "../projects/Ambuja/DSM_cloudoptimised.tif",
  orthoUrl: "../projects/Ambuja/Ortho_CO.tif",
  extent: [/* Paste extent from python here */],
  zOffset: /* Paste zOffset from python here */
}
```

Step 4: Deploy the Code

Now that `ser.js` knows about the new project, you need to upload the updated code to S3.

1. In your terminal, run this exact command:

```
npm run deploy
```

(This command automatically builds the code and pushes it to S3).

Step 5: Add the Link to your Database

1. Take the base URL of your viewer and append `?project=` followed by the exact name you used in Step 3.
Example: `https://polygon-giro3d.s3.ap-south-2.amazonaws.com/index.html?project=ambuja_project`
2. Save this URL in your Polygon MySQL database.
3. When your Angular app loads, it will pass this URL to the iframe, and the new project will render instantly!

How to Update the Viewer Code

If you ever need to change the UI (e.g., editing `style.css` to change a button color) or update the logic in `ser.js`, you do **not** need to touch the database or the `.tif` files!

The Deployment Command

Because web browsers aggressively cache websites to save speed, simply uploading new code isn't enough. You have to tell the browser to clear its cache.

To solve this, we created a single automated command. After you finish writing your code updates, just open your terminal and run:

```
npm run deploy
```

What this command does automatically:

1. Runs `vite build` to package your new code.
2. Uses the AWS CLI to sync your new Javascript and CSS to S3.
3. Uploads `index.html` with a special `--cache-control "no-cache"` rule attached to it.

Because `index.html` is never cached, your users will **always** download your newest code instantly the second they refresh the page! You never have to manually log into AWS to update the website.